

RAG-Projekt — Architektur

Aufbau, Schichten und Datenfluss

Überblick

Das Projekt ist eine Python-Anwendung für Retrieval-Augmented Generation (RAG). Der Quellcode liegt als installierbares Paket unter `src/rag_project/` und ist in fünf Schichten gegliedert: eine Konfigurations- und Datenschicht (`core`), eine Modell- und Zugriffsschicht (`llm`), eine Vorverarbeitungs-Pipeline (`pipeline`), die Anwendungs- und CLI-Schicht (`app`) sowie kleine Hilfsfunktionen (`utils`). Diese Gliederung ist die zentrale Änderung gegenüber früheren Fassungen: Die Module liegen nicht mehr flach im Paketwurzelvezeichnis, sondern in ihren Fachordnern.

Verzeichnisstruktur

Die folgende Übersicht zeigt den Quellbaum, wie er aus dem Projekt selbst hervorgeht.

Pfad	Inhalt
<code>src/rag_project/__init__.py</code>	Paket-API: <code>main</code> , <code>build_parser</code> , <code>Settings</code> , <code>Provider</code> , RAG-Pipeline
<code>src/rag_project/app/</code>	CLIs und Anwendungslogik: <code>main</code> , <code>main_chatbot</code> , <code>main_chunking</code> , <code>main_cleaning</code> , <code>main_loading</code> , <code>rag_app</code> , <code>rag_lib</code> , <code>rag_utils</code>
<code>src/rag_project/core/</code>	<code>config.py</code> (<code>Settings</code> , <code>Keys</code> , <code>Pfade</code>), <code>models.py</code> (Modellkatalog), <code>paths.py</code> (Quelldokumente auflösen)
<code>src/rag_project/llm/</code>	<code>providers.py</code> (Client bauen, Ziel auflösen, Key entschlüsseln)
<code>src/rag_project/pipeline/</code>	<code>loading.py</code> , <code>cleaning.py</code> , <code>chunking.py</code> — die drei Vorverarbeitungsstufen
<code>src/rag_project/utils/</code>	<code>math_utils.py</code> — Rechenhelfer
<code>tests/</code>	Pytest-Suite, neun Testdateien
<code>docs/</code>	Wissensbasis: <code>handbuch.pdf/.txt (de)</code> , <code>handbook.pdf/.txt (en)</code>

Schichten und Verantwortlichkeiten

Jede Schicht kennt nur die Schicht unter sich. Die Anwendungsschicht ruft die Pipeline, die Pipeline ruft die Core-Bausteine, und der Zugriff auf Umgebungsvariablen ist auf genau eine Stelle begrenzt: core/config.py. Kein anderes Modul liest selbst aus os.environ.

Schicht	Aufgabe	Öffentliche Bausteine
core.config	Umgebungsvariablen laden, Provider wählen, Keys als SecretStr halten, Pfade auflösen	Settings, get_settings, load_environment, describe_settings, ChatProvider
core.models	Modellkatalog je Anbieter, Auflösung von Kennungen	ModelInfo, ModelKind, ModelRegistry, registry, format_catalog
core.paths	Quelldokumente finden — CLI-Argument vor .env vor Fallback	resolve_documents, describe_paths, DocumentPathError
llm.providers	Aus Settings und Katalog einen fertigen Client bauen	resolve_target, build_llm, provider_report, ResolvedTarget
pipeline.loading	Dateien und Netzpunkte zu LangChain-Dokument-Objekten	load_documents, detect_format, detect_source_kind, to_jsonl, from_jsonl
pipeline.cleaning	Steuerzeichen, Unicode, Silbentrennung, Leerraum bereinigen	clean_documents, clean_text, CleaningStats
pipeline.chunking	Dokumente in überlappende Stücke teilen, Metadaten anreichern	split_documents, compare_strategies, build_splitter, ChunkingStats
app.rag_lib	Index, Retriever und RAG-Kette über den drei Stufen	run_pipeline, create_rag_system, build_vectorstore, build_rag_chain

Datenfluss

Ein Lauf der RAG-Anwendung durchläuft fünf Stufen. Die ersten drei sind rein lokal und brauchen weder Netz noch Zugangsdaten; erst die Stufen vier und fünf sprechen mit einem Anbieter.

- laden — pipeline/loading.py liest die Quelldatei und liefert je Seite ein Document mit Metadaten (source, file_name, format, size_bytes, page_index).
- bereinigen — pipeline/cleaning.py entfernt Steuerzeichen, normalisiert Unicode, löst Silbentrennung auf und ergänzt chars_before / chars_after.
- chunken — pipeline/chunking.py schneidet die bereinigten Texte in überlappende Stücke und schreibt chunk_index, chunk_chars, chapter, section und title in die Metadaten.
- indexieren — app/rag_lib.py baut einen Qdrant-Store im Arbeitsspeicher (location=':memory:') aus den Chunks.
- antworten — app/rag_lib.py formt aus Treffern einen Kontextblock mit Herkunftsangabe und schickt ihn in die RAG-Kette.

Reihenfolge ist bindend

- Erst bereinigen, dann teilen. Wer die Schritte vertauscht, trägt Steuerzeichen und doppelte Leerzeichen über die Chunk-Grenzen hinweg in die Embeddings.

Konfiguration und Pfade

Alle Einstellungen kommen über get_settings() aus core/config.py. Die Auflösungsreihenfolge ist Argument, dann Umgebungsvariable, dann .env, dann Standardwert. Die Projektwurzel wird nicht gezählt, sondern gesucht: _find_project_root() steigt vom Modulpfad aufwärts, bis es eine pyproject.toml findet. Verschiebt sich das Paket im Baum, findet die Suche die Wurzel trotzdem.

Die Quelldokumente löst core/paths.py auf. Die Priorität ist: Pfade von der Kommandozeile, dann DOCUMENT_FILES aus der .env, dann DOCS_DIR und DEFAULT_DOCUMENT, schließlich der Fallback docs/handbuch.txt relativ zur Projektwurzel.

Fehlerbehandlung

Fehler tragen eigene Typen, damit die CLIs sie in Exit-Codes übersetzen können, statt einen Stacktrace auszugeben. LoaderDependencyError bedeutet ein fehlendes Paket und ergibt Exit-Code 2, UnsupportedFormatError eine unbekannte Endung und Exit-Code 1, DocumentPathError eine fehlende Quelldatei und Exit-Code 1.

Fehlertyp	Modul	Bedeutung
DocumentPathError	core/paths.py	Kein Quelldokument auffindbar
UnsupportedFormatError	pipeline/loading.py	Endung wird nicht unterstützt
LoaderDependencyError	pipeline/loading.py	Ein Loader-Paket fehlt

Fehlertyp	Modul	Bedeutung
UnknownModelError	core/models.py	Modell nicht im Katalog
ProviderError	llm/providers.py	Modell und aktiver Provider passen nicht zusammen
MissingCredentialsError	llm/providers.py	Key des gewählten Providers fehlt
IndexBuildError	app/rag_lib.py	Vectorstore ließ sich nicht aufbauen

Erweiterungspunkte

Ein neues Modell braucht genau eine Zeile: einen Eintrag in RAW_MODELS in core/models.py, gepflegt ohne Provider-Präfix. Das Präfix wird beim Auflösen ergänzt. Eine neue Dateiendung braucht einen Eintrag in der Format-Erkennung in pipeline/loading.py. Eine neue Sprache in der RAG-Anwendung braucht einen Eintrag in SOURCE_FILES und SYSTEM_PROMPTS in app/rag_lib.py.